

Note: This worksheet builds on your solutions for the previous worksheet (**Introducing Classes**)

A) Constructors: Customising Initialisation

Exercise A1) Adding a parameterised constructor to Person

Add a parameterised constructor to your existing Person class. This constructor should take in all attributes of the Person and store them

Exercise A2) Using your parameterised constructor

Amend your previous program in which the user creates a Person to use the parameterised constructor to create and fill the object in one action, rather than creating a default Person and then filling it with values one at a time.

Commit your code when your constructor has been added & you are creating a Person object with initial parameters.

Exercise A3) Adding a constructor that only takes in a subset of parameters to Rectangle

Add two constructors to your Rectangle class:

Constructor one **takes in all components of a Rectangle (length, width, colour)** as parameters and saves them within the new Rectangle.

Constructor two **takes in only the length and width of a Rectangle** as parameters. Your constructor should save the supplied values and should set the colour attribute to "Blue" (i.e. all Rectangles made this way should be created as Blue)

Exercise A4) Using your new constructor

Amend your previous program in which the user creates a list of 5 Rectangles to use the parameterised constructor to create and fill the object in one action, rather than creating a default Person and then filling it with values one at a time. To decide if you should ask the user for a colour or use the default colour value, your program should generate a random number between 1 and 10.

- If the number is odd, the default colour should be used.
- If the number is even, the user should be prompted to enter a colour.

Useful hints:

- `Random rg = new Random()` will create a random number generator.
- `rg.nextInt(10)` will generate a random number between 0 and 9.

Commit your code after each Rectangle constructor is added and you are creating Rectangles with user-specified parameters.

B) Encapsulation: Locking down access to attributes

Exercise B1) Employee

For the following exercise, commit your code after each method has been completed.

1. Create an **Employee** class in the **entities** package. An Employee should contain 5 attributes (variables):
 - a. First name (String)
 - b. Last name (String)
 - c. Id (int)
 - d. Salary (double)
 - e. Job title (String)

All variables should be **private**. There should be no default values assigned to these variables, i.e. you should declare the variables but not set them.

2. Add a **parameterised constructor** to the **Employee** class that takes in and stores values for all attributes.
Note: You cannot test this method yet, as there is no way to view the private content outside the class yet.
3. Add a second **parameterised constructor** to the **Employee** class that takes only the ID value as a parameter.
 - a. The first name, last name, job title and salary attributes should be set to default values. These can be any values of your choosing.

Note: You cannot test this method yet, as there is no way to view the private content outside the class yet.

4. Add the following getter methods to the **Employee** class:

- a. `getFirstName()` should return the first name attribute
- b. `getLastName()` should return the last name attribute
- c. `getSalary()` should return the salary attribute
- d. `getId()` should return the id attribute
- e. `getJobTitle()` should return the job title attribute

Test these in a **program** (i.e. a class containing a `main()` method) called **EmployeeTestBed** in the **apps** package. This should be done by creating an `Employee` and printing the result of each getter call.

5. Add a **display()** method to the class.
 - a. This display should take the format:
`Employee{id: W, first name: X, last name: Y, salary: Z}`
Where W, X, Y and Z are the values stored within the various attributes.

Test this in your **EmployeeTestBed** program. This should be done by calling `display()` on the employee you created when testing `getSalary()`.

6. Add a **calcNetPay()** method to the class. The method should:
 - a. Calculate the tax owed for the year (42% of the salary)
 - b. Calculate the take-home pay for the year (salary – tax)
 - c. Calculate the monthly take-home pay (yearly take-home pay / 12)
 - d. Return the monthly take-home pay.

Test this in your **EmployeeTestBed** program. This should be done by calling `calcNetPay()` on the employee object you previously created and printing the result.

7. Add a **calcBonus()** method to the class. The method should:
 - a. Check what bonus level is to be applied.
 - i. If the job title contains the word “manager”, a 15% bonus should be applied.
 - ii. If the job title contains the word “intern”, a 2% bonus should be applied.
 - iii. All other job titles should receive a 6% bonus.
8. Test this in your **EmployeeTestBed** program. This should be done by calling `calcBonus()` on the employee object you previously created and printing the result.

Exercise B2)

In a separate file called **EmployeeApp.java** in the **apps** package, write a program called **EmployeeApp** that asks a user to create 5 Employees and stores them in an `ArrayList`. For each one, your program should:

1. Ask the user for this Employee's ID
2. Check if the id provided exists already

Hint: You need to check each of the existing Employees to see if its ID matches the one entered by the user

- a. If the id already exists, ask the user to enter another one. This should repeat until a unique id is provided.
3. Ask the user for the remainder of this Employee's information
4. Use the parameterised constructor to create an `Employee` with that data
5. Add the new `Employee` to your collection

Once your collection contains 5 Employees, your program should:

1. Locate the Employee with the **lowest net pay** and display its details.
2. Locate the Employee with the **highest bonus pay** and display its details.
3. Locate the Employees with the word “manager” in their job title and display their details.
4. Locate the **manager** (i.e. an Employee with the word “manager” in its job title) with the highest bonus pay and display its details.

Commit your code:

- When the user can create employees
- When your program can identify the highest bonus pay and display the Employee with that information.
- When your program can identify the lowest net pay and display the Employee with that information.
- When your program can identify and display the details of all managers in the system.
- When your program can identify and display the manager with the highest bonus pay in the system.